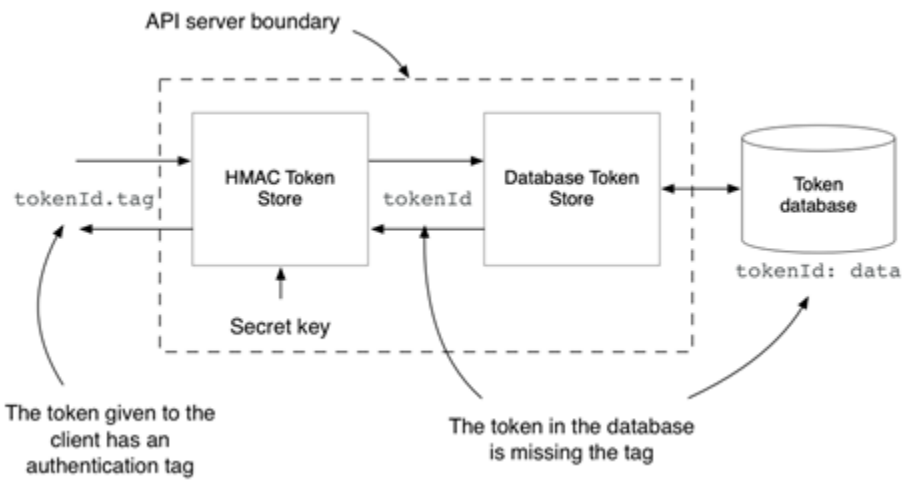


Figure 5.9 The database token ID is left untouched, but an HMAC authentication tag is computed and attached to the token ID returned to API clients. When a token is presented to the API, the authentication tag is first validated and then stripped from the token ID before passing it to the database token store. If the authentication tag is invalid, then the token is rejected before any database lookup occurs.

Listing 5.10 shows the code for computing the HMAC tag and appending it to the token. You can implement this as a new `HmacTokenStore` implementation that can be wrapped around the `DatabaseTokenStore` to add the protections, as HMAC turns out to be useful for other token stores as you will see in the next chapter. The HMAC tag can be implement using the `javax.crypto.Mac` class in Java, using a `Key` object passed to your constructor. You'll see soon how to generate the key. Create a new file HmacTokenStore.java alongside the existing JsonTokenStore.java and type in the contents of listing 5.10.